Pack 02 · Memory and capacity

What fits on a GPU: weights, KV cache, and how many users that leaves room for. Lessons 03–05, 07, 16–19, 25, 42, 50. Answer each question out loud in about a minute, then check the card.

Q1 · Derive KV bytes per token, and what GQA buys SOLID

Lessons: 05, 07, 17

> **INTERVIEWER**
>
> Llama 3 8B has 32 layers, 8 KV heads, head dimension 128, in BF16. How many bytes of KV cache does one token take? Llama 2 7B is similar but has 32 KV heads. What does that do to users per GPU?

60-second answer

- KV per token = **2 (K and V) × layers × KV heads × head dim × bytes**.
- Llama 3 8B: 2 × 32 × 8 × 128 × 2 = 131,072 B = **128 KiB**.
- Llama 2 7B (32 KV heads, no GQA): **512 KiB**, 4× more, so about **4× fewer users** in the same pool.
- GQA shares each K/V head across several query heads, cutting KV memory and the KV bytes read per decode step, with little quality loss.

Key numbers: 8B: 128 KiB per token; 70B (80 layers, 8 KV heads): 320 KiB per token.

Likely follow-up: "How big is one 4,096-token conversation?" 4,096 × 128 KiB = 512 MiB.

> **TRAP**
>
> Name every factor out loud (K and V, layers, KV heads, head dim, bytes). I once gave the exponents without saying what they were.

Q2 · Parameter count and "host MoE like a 13B model" PRIORITY

Lessons: 05, 07

> **INTERVIEWER**
>
> Where do Llama 3 8B's parameters live (h = 4,096, MLP size 14,336, KV width 1,024)? And a teammate says Mixtral 8x7B only uses about 13B parameters per token, so host it like a 13B model on one 80 GB H100 in BF16. Right?

60-second answer

- Per block: Q and O are h × h, K and V are h × 1,024, and the MLP has three h × 14,336 matrices: ≈ 34 M + 8 M + 176 M ≈ **218 M**, and the **MLP is ≈ 81%**.

- 32 blocks ≈ 7 B, plus embeddings and the LM head ≈ 1 B, gives ≈ 8 B.
- MoE: **memory follows total parameters, compute follows active ones**.
- Mixtral stores 46.7 B parameters ≈ **93 GB in BF16**, which doesn't fit in 80 GB. Per token it computes like ≈ 13 B, so it's cheap in FLOPs but not in memory.

Key numbers: block ≈ $2h^2 + 2h \times$ KV width $+ 3h \times$ MLP size.

Likely follow-up: "Does MoE decode read all 93 GB per step?" At large batch, nearly every expert is used by some token, so yes, roughly; at batch 1 only the active experts are read.

> **TRAP**
>
> Finish with the verdict. I stated the concept (memory = total) but never said "93 GB doesn't fit in 80 GB".

Q3 · Does Llama 3 70B fit on one H100? PRIORITY

Lessons: 03, 04, 16, 18, 50

> **INTERVIEWER**
>
> Llama 3 70B runs in BF16 across two H100s (80 GB, 3.35 TB/s each). The PM wants it on one. What are the weights in BF16, FP8 and 4-bit, which can actually serve users, and what's the decode time per token?

60-second answer

- Weights = parameters × bytes: BF16 ≈ **141 GB**, FP8 ≈ **70 GB**, 4-bit ≈ **36 GB** (including the scale factors).
- FP8 **loads but can't serve**: about 10 GB is left for activations, CUDA graphs and KV cache.
- 4-bit leaves ≈ 40 GB for KV, ≈ 120k tokens at 320 KiB per token.
- Decode floor = bytes ÷ bandwidth: BF16 ≈ 42 ms on one GPU (≈ 21 ms across two), FP8 ≈ 21 ms, 4-bit ≈ **10.7 ms**. So 4-bit on one GPU matches BF16 on two.

Key numbers: weights = params × bits ÷ 8; decode floor = weight bytes ÷ bandwidth.

Likely follow-up: "We have A100s. Does FP8 still help?" A100 has no FP8 tensor cores: FP8 weights save memory and decode time, but they're expanded to 16 bits for the math, so prefill doesn't speed up.

> **TRAP**
>
> I first said decode time stays the same with fewer bits. It follows the bytes, so it halves with each halving of bits. And it's bytes ÷ (bytes/s), not "bytes per second ÷ bandwidth".

Q4 · "We can only serve 3 users per GPU?" PRIORITY

Lessons: 19, 25, 42

> **INTERVIEWER**
>
> Llama 3.1 8B on one H100, vLLM defaults. The startup log says "Maximum concurrency for 131,072 tokens per request: 3.43x" and the PM panics. How many 1,800-token chats really fit, and what should max_num_seqs be if the p95 request is 2,400 tokens?

60-second answer

- The log is a **worst case at max_model_len**: the 54.8 GiB pool holds ≈ 449k tokens, and 449k ÷ 131,072 = 3.43.
- Real chats: 449k ÷ 1,800 ≈ **249 at once**.
- Size max_num_seqs from a **high percentile, not the mean**: 449k ÷ 2,400 ≈ 187, then **round down** with margin (≈ 160–180).
- Admitting more than fits at final length causes **preemptions**: silent tail spikes with zero errors.

Key numbers: pool tokens = pool bytes ÷ KV per token; requests that fit = pool tokens ÷ final tokens per request.

Likely follow-up: "An agent product (1,000-token prompts, 6,000-token outputs) has P99 TPOT spikes but no errors, with max_num_seqs at 1,024 and a 533k-token pool." Only 533k ÷ 7,000 ≈ 76 fit at final length; the counter `num_preemptions_total` confirms it. Set max_num_seqs ≈ 70.

> **TRAP**
>
> I confused tokens per request with number of requests ("anything above 2,400"), and wavered on rounding. Always round down: admitting more than fits is what causes preemptions.

Q5 · Three startup crashes, and the utilization cap PRIORITY

Lessons: 18, 42

> **INTERVIEWER**
>
> vLLM crashes on three different days: while loading the model, while sizing the KV cache after setting max_model_len = 1,000,000, and during CUDA graph capture. What ran out each time? Later someone wants to raise gpu_memory_utilization from 0.92 to 0.99. What could go wrong?

60-second answer

- **Loading:** the weights don't fit. Use a smaller precision or more GPUs.
- **Sizing the cache:** one max-length request's KV is bigger than the whole pool. Lower max_model_len; the cache is already everything left over.
- **Graph capture:** the **headroom outside the budget** is too small. Lower gpu_memory_utilization or capture fewer graph sizes.
- Raising 0.92 → 0.95 on 79.1 GiB adds ≈ 2.4 GiB ≈ 19k tokens ≈ 10 chats. Going to 0.99 risks an **out-of-memory crash at runtime under peak load**, after a clean startup.

Key numbers: extra KV tokens = Δutilization × GPU memory ÷ KV per token.

Likely follow-up: "Why is the runtime failure worse than a startup failure?" It passes every test at low load and crashes in production at the peak.

> **TRAP**
>
> For crash 2 I said "reduce the KV cache". The max length is what's too big. For 0.99 I named the headroom but not when it fails (at runtime).

Q6 · fp8 KV cache or 8-bit weights? PRIORITY

Lessons: 42, 50

> **INTERVIEWER**
>
> Same 8B server: 54.8 GiB pool, 128 KiB per token, 15 GiB of weights. You can flip one flag: fp8 KV cache, or 8-bit weights (15 → 7.5 GiB). Which adds more KV capacity, and does the answer change for 70B on two H100s?

60-second answer

- **fp8 KV halves bytes per token**, so it **doubles** the pool: 449k → ≈ 900k tokens.
- **8-bit weights add a fixed amount**, equal to the freed memory: 7.5 GiB ≈ 61k tokens.
- For 8B, fp8 KV wins easily because the pool is large and the weights small.
- For 70B on two H100s, the weights (141 GB) dominate and the pool is small. 8-bit weights free ≈ 70 GB, far more than doubling the small pool, so **8-bit weights win**.

Key numbers: fp8 KV: capacity × 2; weight quantization: + freed bytes ÷ KV per token.

Likely follow-up: "Can you do both?" Yes, and on 70B that's the usual setup: FP8 weights and an FP8 KV cache.

> **TRAP**
>
> I said both flags double capacity. Only fp8 KV doubles it; smaller weights add a fixed amount.